

Midterm Journal

A. The front end, in my own words

The JavaScript fundamentals topics took me back to Web Programming for CS Students, our course from first year. I was rusty going in. I haven't built a web project since then, so relearning variables, functions, arrays, and objects felt less like learning and more like remembering. What finally clicked was arrays and objects working together, once I started using `.map()` on an array of objects instead of looping with indexes by hand. Seeing the data shape change in the console made it concrete instead of abstract.

React, Node, Express, and REST were words I'd already absorbed secondhand, from seeing other developers mention them on social media, without ever having built anything with them myself. If I had to explain a component to someone a year below me, I'd say: a component is a function that returns UI, and you can reuse it the way you'd reuse any function, except a component's output is markup instead of a value. State is not just a variable because a variable change doesn't repaint the screen; state is a variable React is watching, so changing it tells React to re-render. A plain `let` never does that on its own.

For styling, the biggest shift was realizing "deliberate" means every spacing and color choice answers a reason, not a default. Building the `ShelfCarSlot` and `CarPickerModal` components for `PixelRack` was the first time I picked pixel-grid spacing on purpose instead of eyeballing margins.

B. The backend half

This was the hardest part, so I'll be specific. A server "listening" means a Node process is bound to a port and staying alive, waiting for HTTP requests instead of running once and exiting like a script. A route is a rule that says: when a request with this method and this path arrives, run this function.

REST, in my own words: an endpoint is RESTful when the URL names a resource and the HTTP verb names the action on it, instead of the verb being baked into the URL. In

car.routes.js I wrote POST /cars/upload, which runs requirePixelation and multer before the controller, then uploadCar, and it returns the newly created car record as JSON. GET / returns the list of cars for the signed-in user; PATCH /:id and DELETE /:id update or remove one.

For the database, SQL gives you constraints, relationships, and the ability to query across records without loading everything into memory first. A JS array can't enforce that a car belongs to a real user, or ask the database itself to filter and sort. Since PixelRack uses Prisma over Neon Postgres, the query I was proudest of was the one filtering a user's cars by scene while joining in their uploaded sprite. It replaced what would've been three separate array .filter() calls if I'd tried to fake it in memory.

Connecting the two sides surprised me because the front end doesn't know or care that Express exists. It just calls a URL and expects JSON back. Once I understood that, CORS errors stopped being mysterious and started being a sign that the two sides don't trust each other's origin yet.

C. What clicked, and what is still shaky

What clicked: middleware order. Watching requirePixelation run before multer in the upload route, on purpose, so a disabled feature rejects the request before the file is even accepted, made me understand middleware as a pipeline, not decoration.

Still shaky, precisely: why my POST works in Postman but not from my React form. It's almost always a payload or header mismatch (FormData vs. JSON), but I still have to debug it every time instead of predicting it.

Bug I'm proud of: cars weren't showing after upload. The request succeeded, but the list stayed empty. The real cause was that listCars was scoped to req.user.id, but my seed data had a different user id than the one I was logged in as. It had nothing to do with React at all. I found it by logging the query result on the server, not the client, which is now my first move.

Error I now recognize on sight: Cannot read properties of undefined (reading 'map'). It almost always means I'm rendering before the fetched data has arrived.

D. How I work

When I get stuck now, I read the actual error and check the network tab before touching any code. At the start of the term I would just start editing and hope. I do use AI tools, mainly as a second pair of eyes on GitHub Copilot-style corrections while doing the weekly activities. I'd write the attempt myself first, then ask why something failed. A few times it handed me a fix using a library method I couldn't explain, and I made myself look it up before keeping it rather than pasting it in blind. That's the rule I'm keeping. The activity I left to the last minute was the styling pass on the car picker modal, and it cost me a rushed layout I had to redo the next week.

E. Looking forward to the final project

PixelRack leans hardest on the REST and database sections, specifically the car upload route and the Prisma schema, which is what the activity on building an authenticated CRUD API taught me directly. The biggest gap between what I can build today and what the plan needs is confident error handling across async routes; right now I patch it reactively instead of designing for it. Two weeks from now, concretely: the scene page will persist car placement to the database instead of only local state, verified by refreshing the page and seeing the layout survive.